C++ Templates

- C++ Function Templates
- C++ Class Templates

C++ Function Templates

- Approaches for functions that implement identical tasks for different data types
  - Naïve Approach
  - Function Overloading
  - Function Template
- Instantiating a Function Templates

Approach 1: Naïve Approach

- create unique functions with unique names for each combination of data types
  - difficult to keeping track of multiple function names
  - lead to programming errors

Example

```
void PrintInt( int n )
{
   cout << "***Debug" << endl;
   cout << "Value is " << n << endl;
}
void PrintChar( char ch )
{
   cout << "***Debug" << endl;
   cout << "Value is " << ch << endl;
}
void PrintFloat( float x )
{
  ...
}
void PrintDouble( double d )
{
  ...
}
```

To output the traced values, we insert:

```
PrintInt(sum);

PrintChar(initial);

PrintFloat(angle);
```

Approach 2:Function Overloading (Review)

- The use of the same name for different C++ functions, distinguished from each other by their parameter lists
  - Eliminates need to come up with many different names for identical tasks.
  - Reduces the chance of unexpected results caused by using the wrong function name.

Example of Function Overloading

```
void Print( int n )
{
   cout << "***Debug" << endl;
   cout << "Value is " << n << endl;
}
void Print( char ch )
{
   cout << "***Debug" << endl;
   cout << "Value is " << ch << endl;
}
void Print( float x )
{

}
```

To output the traced values, we insert:

```
Print(someInt);
Print(someChar);
Print(someFloat);
```

Approach 3: Function Template

- A C++ language construct that allows the compiler to generate <u>multiple</u> versions of a function by allowing <u>parameterized data types</u>.

FunctionTemplate

```
Template < TemplateParamList >
FunctionDefinition
```

TemplateParamDeclaration: placeholder

```
{ class  typeIdentifier
  typename variableIdentifier
```

Example of a Function Template

```
template<class SomeType>

void Print( SomeType val )
{
    cout << "***Debug" << endl;
    cout << "Value is " << val << endl;
}
```

Template parameter

(class, user defined type, built-in types)

Template argument

To output the traced values, we insert:

```
Print<int>(sum);
Print<char>(initial);
Print<float>(angle);
```

Instantiating a Function Template

- When the compiler instantiates a template, it substitutes the template argument for the template parameter throughout the function template.

TemplateFunction Call

```
Function < TemplateArgList > (FunctionArgList)
```

A Function Template - 1

```
// function template
#include <iostream>
using namespace std;

template <class T>
T GetMax (T a, T b) {
  T result;
  result = (a>b)? a : b;
  return (result);
}

int main () {
  int i=5, j=6, k;
  long l=10, m=5, n;
  k=GetMax<int>(i,j);
  n=GetMax<long>(l,m);
  cout << k << endl;
  cout << n << endl;
  return 0;
}
```

A Function Template - 2

```
1   //Program to demonstrate a function template.
2   #include <iostream>
3   using std::cout;
4   using std::endl;

5   //Interchanges the values of variable1 and variable2.
6   //The assignment operator must work for the type T.
7   template<class T>
8   void swapValues(T& variable1, T& variable2)
9   {
10        T temp;

11        temp = variable1;
12        variable1 = variable2;
13        variable2 = temp;
14  }
```

```
15  int main( )
16  {
17      int integer1 = 1, integer2 = 2;
18      cout << "Original integer values are "
19           << integer1 << " " << integer2 << endl;
20      swapValues(integer1, integer2);
21      cout << "Swapped integer values are "
22           << integer1 << " " << integer2 << endl;

23      char symbol1 = 'A', symbol2 = 'B';
24      cout << "Original character values are: "
25           << symbol1 << " " << symbol2 << endl;
26      swapValues(symbol1, symbol2);
27      cout << "Swapped character values are: "
28           << symbol1 << " " << symbol2 << endl;
29      return 0;
30  }
```

Exercise:

write a full C++ program using the function template shown on slide 8, passing to the function three different data types

Summary of Three Approaches

Naïve Approach
Different Function Definitions
Different Function Names

Function Overloading
Different Function Definitions
Same Function Name

Template Functions
One Function Definition (a function template)
Compiler Generates Individual Functions

Class Template

- A C++ language construct that allows the compiler to generate <u>multiple</u> versions of a class by allowing <u>parameterized data types</u>.

Class Template

```
Template < TemplateParamList >
ClassDefinition
```

TemplateParamDeclaration: placeholder

```
class  typeIdentifier
typename variableIdentifier
```

Example of a Class Template

```
template<class ItemType>
class GList
{
 public:
   bool IsEmpty() const;
   bool IsFull() const;
   int  Length() const;
   void Insert( /* in */ ItemType item );
   void Delete( /* in */ ItemType item );
   bool IsPresent( /* in */ ItemType item ) const;
   void SelSort();
   void Print() const;
   GList();                // Constructor
 private:
   int     length;
   ItemType data[MAX_LENGTH];
};
```

Template parameter

Instantiating a Class Template

- Class template arguments *must* be explicit.
- The compiler generates distinct class types called template classes or generated classes.
- When instantiating a template, a compiler substitutes the template argument for the template parameter throughout the class template.

Instantiating a Class Template

To create lists of different data types

```
// Client code

GList<int> list1;
GList<float> list2;
GList<string> list3;

list1.Insert(356);
list2.Insert(84.375);
list3.Insert("Muffler bolt");
```

template argument

Compiler generates 3 distinct class types

```
GList_int list1;
GList_float list2;
GList_string list3;
```

Substitution Example

```
class GList_int
{
public:

void Insert( /* in */ ItemType item );

   void Delete( /* in */ ItemType item );

   bool IsPresent( /* in */ ItemType item ) const;

private:
   int     length;
   ItemType data[MAX_LENGTH];
};
```

int

int

int

int

Function Definitions for Members of a Template Class

```
template<class ItemType>
void GList<ItemType>::Insert( /* in */ ItemType item )
{
   data[length] = item;
   length++;
}
```

```
//after substitution of float
void GList<float>::Insert( /* in */ float item )
{
   data[length] = item;
   length++;
}
```

Another Template Example: passing two parameters

```
template <class T, int size>
  class Stack {...
  };

Stack<int,128> mystack;
```

non-type parameter

Class Template

```
// class templates
#include <iostream>
using namespace std;

template <class T>
class mypair {
    T a, b;
  public:
    mypair (T first, T second)
      {a=first; b=second;}
    T getmax ();
};

template <class T>
T mypair<T>::getmax ()
{
  T retval;
  retval = a>b? a : b;
  return retval;
}

int main () {
  mypair <int> myobject (100, 75);
  cout << myobject.getmax();
  return 0;
}
```